

Laboratorio di Programmazione

(Laurea triennale in matematica)

Lezione 10

Usare le funzioni in C

Per poter essere utilizzata, una funzione deve essere:

- **dichiarata** (prima del **main**) fornendo il cosiddetto **prototipo** *di funzione*;
- **invocata** in un qualsiasi punto del programma in cui è utile.
- **definita** (anche dopo il **main**) come successione di istruzioni.
- **ATTENZIONE**: In C non si possono definire funzioni dentro funzioni.

Dichiarazione di una funzione

Sintassi di dichiarazione, detta **prototipo** (*prototyping*):

tipoRitorno nomeFunz(tipoPar1, ..., tipoParN) ;

Tipo ritornato dalla funzione, nome della funzione.

Tra parentesi: tipi dei parametri in ingresso separati da virgola.

Si mette di solito dopo l'inclusione delle librerie

Presuppone esista da qualche parte la definizione della funzione.

ESEMPI:

- **double cubo(float)**

Tipo ritornato **double**, nome **cubo**, un parametro di tipo **float**.

- **int max(int,int,int)**

Tipo ritornato **int**, nome **max**, tre parametri di tipo **int**.

Definizione di una funzione

Sintassi di definizione:

```
tipoRitorno nomeFunz(tipoPar1 par1,...,tipoParN parN)  
  
{      dichiarazioni di variabili;  
        istruzioni...  
        return (valore o espressione);  
  
}
```

- Tipo ritornato dalla funzione (se omesso viene considerato **int**)
- Nome della funzione.
- Tra parentesi tonde: tipo e nome dei parametri in ingresso – ***parametri formali*** – separati da virgola.

ATTENZIONE: Ciascun parametro deve essere preceduto dal suo tipo (il tipo va ripetuto anche se ci sono più parametri dello stesso tipo)

- Dichiarazioni di variabili (***variabili locali***)
- Blocco d'istruzioni che termina con **istruzione return**

Definizione di una funzione (esempi)

```
double cubo(float c) {  
    return (c*c*c);  
}
```

```
double cubo(float c) {  
    double risul;  
  
    risul=c*c*c;  
    return risul;  
}
```

Definizione di una funzione (esempi)

```
int maxTest(int a, int b, int c) {  
    int max;  
    max=a;  
    if (b>max) max= b;  
    if (c>max) max= c;  
    return max;  
}
```

Definizione di una funzione (esempi)

```
int PrimoTest ( int num) {  
    int primo = 1;  
    int d = 2;  
    int fine = num/2;  
    while ( d <= fine && primo==1) {  
        if ( num % d == 0 )  
            primo = 0;  
        d++;  
    }  
    return primo;  
}
```

Variabili di una funzione

- Le variabili utilizzabili sono:
 - ***parametri formali*** (tra parentesi tonde)
 - ***variabili locali*** (date dalle dichiarazioni di variabili)
 - ***variabili globali*** (definite fuori da ogni funzione)
- ATTENZIONE: i parametri formali e le variabili locali esistono solo alla chiamata della funzione. Dopo l'istruzione return vengono "persi". Varie chiamate della funzione generano "diverse copie" di tali variabili.

Funzione scambio ?

```
void scambia(int, int)
```

```
main() {
```

```
    int x,y;
```

```
    x=5; y=10;
```

```
    scambia(x,y);
```

```
    printf("x=%d, y=%d",x,y);
```

```
}
```

```
void scambia(int x, int y) {
```

```
    int aux;
```

```
    aux = x;
```

```
    x = y;
```

```
    y = aux;
```

```
}
```

☹ ...Non così!
userò i puntatori..

Gestione della memoria durante l'esecuzione

Codice macchina e dati vengono memorizzati entrambi nella RAM, ma in zone separate:

- Memoria per il codice macchina fissato a tempo di compilazione
- Memoria per dati locali alle funzioni (variabili e parametri) che cresce e decresce dinamicamente durante l'esecuzione(di tipo stack: struttura dati con accesso LIFO: Last In First Out)
- Durante l'esecuzione del programma viene gestita la pila dei record di attivazione (RDA):
- per ogni attivazione di funzione viene allocato un RDA in cima allo stack
- al termine dell'attivazione della funzione il RDA viene rimosso

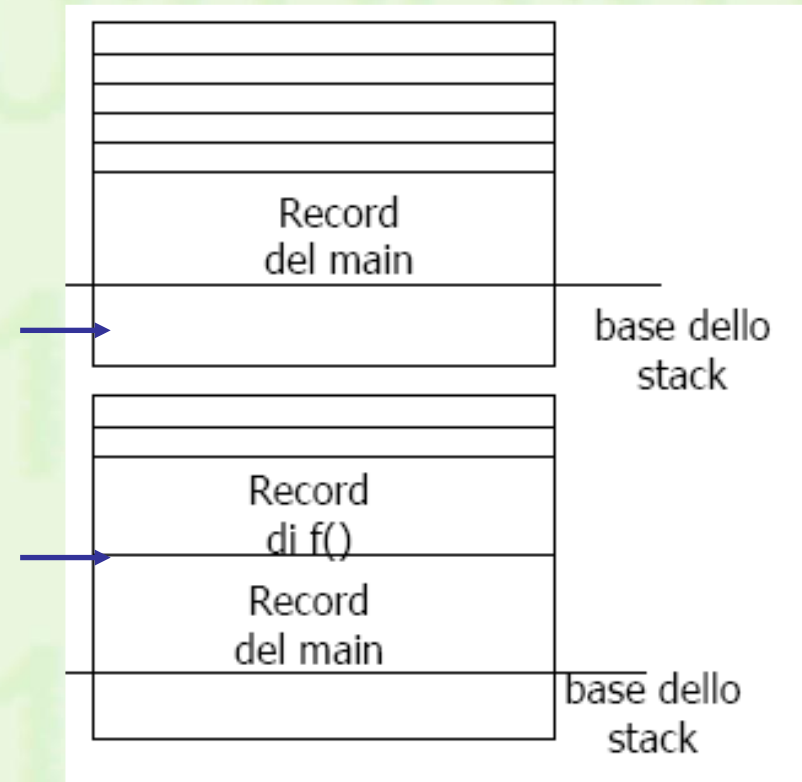
Record di Attivazione

Ogni RDA contiene:

- Le locazioni di memoria dei parametri formali;
- Le locazioni di memoria per le variabili locali;
- L'indirizzo di ritorno: indirizzo della prossima istruzione della funzione chiamante.

Record di attivazione

- ❑ Quando un programma viene lanciato, l'unica funzione attiva è il main. Nello stack è riservato spazio per le variabili locali al main
- ❑ Se durante l'esecuzione di main() viene chiamata la funzione f() viene allocato un nuovo record di attivazione



- Il record di attivazione di f() contiene anche l'indirizzo di ritorno (IDR) all'istruzione del main successiva alla chiamata.

Esempio

```
#include <stdio.h>
double quad(float); double cubo(float);
double quar(float); double quin(float);
double pote(float, int);
int main(void){
    int base, esponente; double ptnz;
    printf(" Inserire base: " );
    scanf("%d", &base);
    printf(" Inserire esponente (0-5): ");
    scanf("%d", &esponente);
    ptnz = pote( base, esponente);
    if (ptnz == -1)
        printf("Potenza non prevista\n");
    else
        printf("La potenza %d di %d e' %f\n",
                esponente, base, ptnz);
}
```

continua ...

... segue

```
double quad(float c){return(c*c);}  
double cubo(float c){return(c*c*c);}  
double quar(float c){return(c*c*c*c);}  
double quin(float c){return(c*c*c*c*c);}
```

```
double pote(float b, int e){  
switch (e) {  
    case 0: return (1);  
    case 1: return (b);  
    case 2: return (quad( b ));  
    case 3: return (cubo( b ));  
    case 4: return (quar( b ));  
    case 5: return (quin( b ));  
    default : return (-1);  
}}  
FINE
```

```
main
{
...
    ptnz = pote5(base, 2esponente); ← 25
...
}

double pote5(float b, int 2e)
{
    switch (e) {
        ...
        case2: return quad5(b); ← 25
        ...
    }
}

double quad5(float c)
{
    return ( c*c ); ← 25
}
```

The diagram illustrates the execution flow of three functions: `main`, `pote`, and `quad`.
1. In `main`, the call `pote(base, esponente)` is shown. The variable `base` is annotated with the value 5, and `esponente` with 2. An arrow points from this call to the `pote` function, with a label 25 indicating the result of the function call.
2. The `pote` function is defined as `double pote(float b, int e)`. It contains a `switch` statement. The `case2` branch returns `quad(b)`. The variable `b` is annotated with 5. An arrow points from this return statement to the `quad` function, with a label 25 indicating the result of the function call.
3. The `quad` function is defined as `double quad(float c)`. It returns `c*c`. The variable `c` is annotated with 5. An arrow points from the `return` statement to the `case2` branch in `pote`, with a label 25 indicating the result of the function call.

La libreria matematica:

`#include<math.h>`

- La libreria matematica fornisce molte delle funzioni matematiche
- La maggior parte di funzioni restituisce un **double**
- Gli argomenti possono essere: costanti, variabili o espressioni
- Esempi:

```
printf( "%.2f", sqrt( 900.0 ) );
```

```
printf( "%.2f", sqrt( c1 + d * f ) );
```

Function	Description	Example
<code>sqrt(x)</code>	square root of x	<code>sqrt(900.0)</code> is 30.0 <code>sqrt(9.0)</code> is 3.0
<code>cbrt(x)</code>	cube root of x (C99 and C11 only)	<code>cbrt(27.0)</code> is 3.0 <code>cbrt(-8.0)</code> is -2.0
<code>exp(x)</code>	exponential function e^x	<code>exp(1.0)</code> is 2.718282 <code>exp(2.0)</code> is 7.389056
<code>log(x)</code>	natural logarithm of x (base e)	<code>log(2.718282)</code> is 1.0 <code>log(7.389056)</code> is 2.0
<code>log10(x)</code>	logarithm of x (base 10)	<code>log10(1.0)</code> is 0.0 <code>log10(10.0)</code> is 1.0 <code>log10(100.0)</code> is 2.0
<code>fabs(x)</code>	absolute value of x as a floating-point number	<code>fabs(13.5)</code> is 13.5 <code>fabs(0.0)</code> is 0.0 <code>fabs(-13.5)</code> is 13.5
<code>ceil(x)</code>	rounds x to the smallest integer not less than x	<code>ceil(9.2)</code> is 10.0 <code>ceil(-9.8)</code> is -9.0

Function	Description	Example
<code>floor(x)</code>	rounds x to the largest integer not greater than x	<code>floor(9.2)</code> is 9.0 <code>floor(-9.8)</code> is -10.0
<code>pow(x, y)</code>	x raised to power y (x^y)	<code>pow(2, 7)</code> is 128.0 <code>pow(9, .5)</code> is 3.0
<code>fmod(x, y)</code>	remainder of x/y as a floating-point number	<code>fmod(13.657, 2.333)</code> is 1.992
<code>sin(x)</code>	trigonometric sine of x (x in radians)	<code>sin(0.0)</code> is 0.0
<code>cos(x)</code>	trigonometric cosine of x (x in radians)	<code>cos(0.0)</code> is 1.0
<code>tan(x)</code>	trigonometric tangent of x (x in radians)	<code>tan(0.0)</code> is 0.0

Generazione di numeri random

`#include<stdlib.h>`

- PREMESSA: generare numeri casuali è impossibile, perciò parliamo di numeri pseudocasuali.
- `int rand(void) ;`
- ogni chiamata di `rand()` genera e restituisce un intero nell'intervallo `[0, RAND_MAX]`.
- `RAND_MAX` dipende dalla macchina, ma è tipicamente 32767.
- La sequenza prodotta da `rand` è sempre la stessa (pseudocasuale).

Generazione di numeri random

```
#include<stdlib.h>
```

```
#include<time.h>
```

- `void srand(unsigned x) ;`

srand() inizializza il generatore di numeri casuali, in modo che più sequenze generate da rand() inizino da un valore diverso. (Se non specificato, rand() agisce come se fosse stata chiamata srand(1).)

- `srand(time(NULL)) ;`

si utilizza per inizializzare il generatore casuale con un valore differente a ogni esecuzione del programma

- Per modificare l'intervallo dei valori prodotti da **rand()** uso operatore %
- Esempi:

```
/* lancio un dado */  
  
printf("%d \n ", 1 + (rand() % 6));  
  
  
  
  
  
  
/* genero un numero tra -10 e 10*/  
  
x=rand()%21 -10
```